

Galaxy-X-os — One-Page Technical Report

SCALE x ODYSSEY | Deep-learning classification of raw astronomical images into 5 celestial categories. **GitHub:** <https://github.com/Srujan0798/Galaxy-X-os>

Dataset Sources

- **SDSS DR17** (Sloan Digital Sky Survey, Data Release 17) — real astronomical imaging used for the final trained model.
- Merged Kaggle galaxy-morphology datasets (Galaxy Zoo / DeepSky / Planetary imagery) for class balancing.
- **5 classes:** Spiral Galaxy, Elliptical Galaxy, Nebula, Star Cluster, Planetary Object.
- **Split:** stratified **disjoint** 80 / 10 / 10 — **1600 train / 200 val / 200 test**. Verified by MD5 hash that no image appears in more than one split (zero train/test leakage).

Model Architecture

- **Backbone:** EfficientNet-B3 (ImageNet pretrained, transfer learning) — ~11.6M parameters.
- **Head:** Replaced classifier → 5-class output.
- **Training strategy:** Progressive unfreezing (backbone frozen first 3 epochs), mixed-precision (`torch.amp`), AdamW (lr `3e-4`), batch size 32, OneCycleLR scheduler.
- **Augmentations:** Albumentations astro-specific pipeline (cosmic-ray simulation, vignetting, Poisson noise, flips/rotations, label smoothing 0.1).
- **Explainability:** Grad-CAM heatmaps over predictions.

Final Test Metrics

Evaluated on the held-out **disjoint** test set (200 images, 40 per class — none seen in training). Macro-averaged.

Metric	Standard	+ TTA (6x aug)
Accuracy	72.0%	74.5%
Precision (macro)	0.716	—
Recall (macro)	0.720	—
F1 (macro)	0.711	0.741

Per-class F1 (Standard):

Class	F1
Elliptical Galaxy	1.000
Nebula	1.000
Planetary Object	0.660
Star Cluster	0.400
Spiral Galaxy	0.494

Strongest on Elliptical/Nebula (near-perfect); Spiral vs Star Cluster confusion drives the lower macro score (visually similar diffuse structures). TTA adds ~+2.5% accuracy.

Confusion Matrix

See [results/confusion_matrix.png](#). Additional plots: [results/per_class_metrics.png](#), [results/training_curves.png](#), [results/confidence_distribution.png](#).

Inference Time

- **~410 ms per image on Apple MPS** (EfficientNet-B3, mixed precision, batch=1, after warmup).
- **Target <15 ms on CUDA GPU** — code supports `torch.autocast("cuda")` for GPU deployment.
- Streamlit demo runs in real time.

Setup Instructions

```
conda create -n scale_odyssey python=3.10 -y
conda activate scale_odyssey
pip install -r requirements.txt

python src/download_datasets.py # prepare data
python src/train.py             # train
python src/evaluate.py          # evaluate (standard + TTA)
python src/gradcam.py           # Grad-CAM report
streamlit run app/app.py        # web demo
```

Training Script

- **Full pipeline:** [src/train.py](#) — progressive unfreezing + OneCycleLR (for CUDA GPUs).
- **Memory-safe trainer:** [src/train_head.py](#) — frozen backbone + head, used for the reported result on 8 GB hardware.
- **Disjoint split:** [src/generate_splits.py](#). Config: [configs/config.yaml](#). Checkpoint: `checkpoints/best_model.pth`.

Hardware note: trained on an 8 GB MacBook Air (Apple MPS, no CUDA). Full backbone fine-tuning is memory-bound (needs >12 GB and swaps to disk), so the reported result uses transfer

learning with the backbone frozen and the classifier head trained. On a CUDA GPU the same `src/train.py` pipeline supports full fine-tuning and faster (tens-of-ms) inference.